

Final Project Guidelines – Python + MySQL (PDBC)

General Project Guidelines

- The project must be developed using **Python and MySQL Database**.
- Use **PDBC (mysql-connector-python)** for database connectivity.
- The application should be **menu-driven** (console-based).
- Follow **modular programming** using functions and/or classes.
- Use proper **database design (tables, keys, relationships)**.
- Implement **CRUD operations** (Create, Read, Update, Delete).
- Include at least **one JOIN query** and **one aggregate function**.
- Use **exception handling (try-except)** for database operations.
- Apply **transactions (commit & rollback)** wherever required.
- Display output in a **clear and formatted manner**.
- Maintain proper **naming conventions and code readability**.
- Submit:
 - Source Code
 - Database Schema (Tables)
 - Sample Output Screens
 - Project Report

Note:

- Focus on **understanding logic**, not just code
- Ensure your project is **working end-to-end**
- Be prepared to **explain your code and queries**

Project List with Suggested Functions

1. Employee Management System

This project focuses on building a system to manage employee details in an organization using Python and MySQL. Students are required to create functionalities to add new employees, update their details such as salary and department, delete employee records, and display all employee information. The system should use a database table to store employee data and demonstrate CRUD operations along with search functionality. The application should be menu-driven and display results in a clear format.

Functions:

```
add_employee()  
view_employees()  
update_employee()  
delete_employee()  
search_employee()  
connect_db()
```

2. Student Course Management System

In this project, students are required to design a system to manage students, courses, and their enrollments. The application should allow adding student details, creating courses, and assigning courses to students. It should also display student-course relationships using SQL JOIN queries and calculate average marks for each student. This project helps in understanding relational databases and how multiple tables are connected using keys.

Functions:

```
add_student()  
add_course()  
enroll_student()  
view_student_courses()  
calculate_average_marks()  
connect_db()
```

3. Library Management System

This project involves developing a system to manage books and users in a library. Students should implement features to add new books, issue books to users, return books, and track availability. The system should also calculate fines for late returns. All records should be stored in a database, and the program should provide clear outputs for each operation. This project emphasizes real-world database usage and record tracking.

Functions:

```
add_book()  
issue_book()  
return_book()  
view_books()  
calculate_fine()  
connect_db()
```

4. Banking Transaction System

In this project, students are required to simulate basic banking operations such as account creation, deposit, withdrawal, and fund transfer. The key focus is on implementing transactions using commit and rollback to ensure data consistency. For example, during fund transfer, if one operation fails, the entire transaction should be reversed. This project helps students understand transaction handling and its importance in real-world applications.

Functions:

```
create_account()  
deposit()  
withdraw()  
transfer_funds()  
check_balance()  
connect_db()
```

5. Online Shopping System

This project requires students to build a simple backend system for an online shopping platform. It should allow adding products, viewing product lists, placing orders, and calculating the total bill. The system should store product and order details in a database and use SQL queries to retrieve and display information. This project demonstrates how databases are used in e-commerce applications.

Functions:

```
add_product()  
view_products()  
place_order()  
calculate_bill()  
view_orders()  
connect_db()
```

6. Attendance Management System

In this project, students are required to create a system to track and manage attendance records. The application should allow marking attendance for students, viewing attendance records, and calculating attendance percentage. Additional features like filtering attendance by date can also be included. This project helps students understand data recording and reporting using Python and databases.

Functions:

```
mark_attendance()  
view_attendance()  
calculate_percentage()  
filter_by_date()  
connect_db()
```

7. Hospital Management System

This project involves designing a system to manage patients, doctors, and appointments in a hospital. Students should implement functionalities to add patient and doctor details, assign doctors to patients, and schedule appointments. The system should store all records in a database and allow easy retrieval of information. This project reflects a real-world scenario of managing healthcare data.

Functions:

```
add_patient()
add_doctor()
schedule_appointment()
view_records()
assign_doctor()
connect_db()
```

8. Expense Tracker System

In this project, students will develop a system to track daily expenses. The application should allow users to add expenses, categorize them (e.g., food, travel, shopping), and view total expenses. It should also generate monthly reports and category-wise analysis. This project helps students understand data analysis and reporting using Python and databases.

Functions:

```
add_expense()
view_expenses()
category_analysis()
monthly_report()
connect_db()
```

9. Inventory Management System

This project focuses on managing product inventory in a store or warehouse. Students should create functionalities to add products, update stock levels, view available inventory, and identify low stock items. The system should generate reports based on inventory data. This project helps students understand how businesses manage stock and inventory efficiently.

Functions:

```
add_product()
update_stock()
view_inventory()
low_stock_alert()
generate_report()
connect_db()
```

10. Quiz Application (Database Based)

In this project, students will build a quiz system where questions are stored in a database. The application should load questions, display them to the user, accept answers, evaluate responses, and calculate the final score. It can also store user results in the database. This project demonstrates how databases can be used in interactive applications.

Functions:

```
load_questions()
display_questions()
evaluate_answers()
calculate_score()
store_results()
connect_db()
```